Лабораторная работа № 1

Тема: Подключение Internet Shop API к PostgreSQL

Цель: Создать базу данных PostgreSQL для интернет-магазина, подключить её к Express-приложению через пакет pg, заменить чтение товаров из массива запросами SELECT, самостоятельно создать таблицу categories и реализовать GET /api/categories.

Лабораторная работа № 1

Тема: Подключение Internet Shop API к PostgreSQL

Цель работы: создать базу данных PostgreSQL для интернет-магазина, подключить её к Express-приложению через пакет pg, заменить чтение товаров из массива запросами SELECT; самостоятельно спроектировать таблицу categories и реализовать маршрут GET /api/categories.

В этой лабораторной работе реализуется только чтение данных из PostgreSQL.

- GET /api/products
- GET /api/products/:id
- GET /api/categories

INSERT, UPDATE и DELETE из Node.js будут реализованы в следующих работах. В этой работе INSERT используется только для первоначального заполнения базы данных через SQL-клиент.

1. Проверьте PostgreSQL

Убедитесь, что PostgreSQL Server запущен.

- pgAdmin
- psql
- другой SQL-клиент

2. Создайте пользователя приложения

Подключитесь к PostgreSQL с административными правами и выполните:

```
CREATE USER shop_user
WITH PASSWORD 'college123';
```

Пароль college123 используется только для учебной локальной среды.

3. Создайте базу данных

```
CREATE DATABASE shop_db
OWNER shop_user;
```

В результате должна появиться база shop_db, владельцем которой является shop_user.

4. Подключитесь к shop_db как shop_user

В pgAdmin создайте отдельное подключение к серверу со следующими параметрами:

- Host: localhost
- Port: 5432
- Database: shop_db

- User: shop_user
- Password: college123

Либо используйте psql:

```
psql -h localhost -U shop_user -d shop_db
```

5. Самостоятельное задание: создайте таблицу categories

Самостоятельно составьте и выполните SQL-команду CREATE TABLE categories. Требования: id — целочисленный автоматически генерируемый первичный ключ; name — обязательная строка до 100 символов, названия категорий не должны повторяться. Название таблицы: categories. Готовая команда CREATE TABLE не приводится. Создайте таблицу ДО products, поскольку products будет ссылаться на неё внешним ключом.

Проверка: убедитесь через pgAdmin или SQL-клиент, что таблица существует, id является первичным ключом, name не принимает NULL и одинаковые названия категорий не допускаются.

6. Создайте таблицу products

Вместо текстового поля category таблица products будет хранить внешний ключ category_id.

```
CREATE TABLE products (
  id INTEGER
    GENERATED ALWAYS AS IDENTITY
    PRIMARY KEY,

  name VARCHAR(150)
    NOT NULL,

  price INTEGER
    NOT NULL
    CHECK (price > 0),

  category_id INTEGER
    NOT NULL
    REFERENCES categories(id),

  in_stock BOOLEAN
    NOT NULL
    DEFAULT true
);
```

7. Добавьте тестовые категории

```
INSERT INTO categories (name)
VALUES
  ('Accessories'),
  ('Monitors'),
  ('Components');
```

8. Проверьте таблицу categories

```
SELECT *
```

```
FROM categories
ORDER BY id;
```

Ожидаемый результат:

```
1 | Accessories
2 | Monitors
3 | Components
```

9. Добавьте тестовые товары

В products указываются идентификаторы категорий из таблицы categories.

```
INSERT INTO products (
  name,
  price,
  category_id,
  in_stock
)
VALUES
  (
    'Keyboard',
    15000,
    1,
    true
  ),
  (
    'Mouse',
    8000,
    1,
    true
  ),
  (
    'Monitor',
    85000,
    2,
    false
  );
```

10. Проверьте таблицу products

```
SELECT *
FROM products
ORDER BY id;
```

Должны появиться три товара.

11. Проверьте связь products и categories

Выполните JOIN, чтобы получить название категории вместе с товаром:

```
SELECT
  p.id,
  p.name,
  p.price,
```

```
  c.name AS category,
  p.in_stock
FROM products AS p
JOIN categories AS c
  ON c.id = p.category_id
ORDER BY p.id;
```

Пример результата:

```
1 | Keyboard | 15000 | Accessories | true
2 | Mouse    | 8000  | Accessories | true
3 | Monitor  | 85000 | Monitors    | false
```

12. Откройте Express-проект

Продолжаем существующий проект express-shop.

```
express-shop/
├── controllers/
│   └── productController.js
├── errors/
├── middleware/
├── routes/
│   └── productRoutes.js
├── services/
│   └── productService.js
├── validators/
├── public/
├── app.js
├── package.json
└── package-lock.json
```

13. Установите PostgreSQL-драйвер

```
npm install pg
```

Пакет pg поддерживает Promise, async/await и предоставляет пул соединений.

14. Создайте каталог config

Итоговая часть структуры проекта:

```
express-shop/
├── config/
│   └── db.js
├── controllers/
├── routes/
├── services/
├── app.js
└── package.json
```

15. Создайте .env

В корне проекта создайте файл .env:

```
PGHOST=localhost
PGPORT=5432
PGDATABASE=shop_db
PGUSER=shop_user
PGPASSWORD=college123
```

16. Добавьте .env в .gitignore

```
node_modules/
.env
```

Пароль базы данных не должен попадать в Git-репозиторий.

17. Создайте config/db.js

```
import {
  loadEnvFile
} from "node:process";

import {
  Pool
} from "pg";

loadEnvFile();

export const pool = new Pool({
  host: process.env.PGHOST,
  port: Number(process.env.PGPORT),
  database: process.env.PGDATABASE,
  user: process.env.PGUSER,
  password: process.env.PGPASSWORD
});
```

process.loadEnvFile() загружает .env в process.env. Дополнительный пакет dotenv для этой конфигурации не требуется.

18. Создайте тест подключения

В корне проекта временно создайте test-db.js:

```
import {
  pool
} from "./config/db.js";

try {
  const result = await pool.query(
    "SELECT NOW() AS current_time"
  );

  console.log(
    "PostgreSQL connected:"
  );
```

```
  console.log(
    result.rows[0]
  );
} catch (error) {
  console.error(
    "Database connection error:"
  );

  console.error(
    error.message
  );
} finally {
  await pool.end();
}
```

19. Запустите проверку

```
node test-db.js
```

Ожидается сообщение вида:

```
PostgreSQL connected:
{ current_time: ... }
```

Если возникла ошибка, проверьте PGHOST, PGPORT, PGDATABASE, PGUSER и PGPASSWORD.

20. Удалите временный тест

После успешной проверки файл test-db.js можно удалить.

pool.end() в основном Express-приложении после каждого запроса вызывать не нужно. Pool должен оставаться доступным для следующих HTTP-запросов.

21. Перепишите productService.js

До подключения PostgreSQL источником данных был массив products. Теперь данные читаются из PostgreSQL.

```
import {
  pool
} from "../config/db.js";

export async function getAllProducts() {
  const result = await pool.query(
    `SELECT
      p.id,
      p.name,
      p.price,
      c.name AS category,
      p.in_stock AS "inStock"
     FROM products AS p
     JOIN categories AS c
       ON c.id = p.category_id
     ORDER BY p.id`
  );
```

```
  return result.rows;
}

export async function findProductById(id) {
  const result = await pool.query(
    `SELECT
      p.id,
      p.name,
      p.price,
      c.name AS category,
      p.in_stock AS "inStock"
     FROM products AS p
     JOIN categories AS c
       ON c.id = p.category_id
     WHERE p.id = $1`,
    [id]
  );

  return result.rows[0];
}
```

JOIN позволяет сохранить прежний формат REST API: клиент по-прежнему получает строковое поле category, хотя в таблице products хранится category_id.

22. Обратите внимание на $1

Не объединяйте значение id с SQL-строкой.

```
const sql =
  "SELECT * FROM products WHERE id = " + id;
```

Используйте параметризованный запрос:

```
await pool.query(
  "SELECT * FROM products WHERE id = $1",
  [id]
);
```

Значение пользователя передаётся отдельно от текста SQL.

23. Измените getProducts в productController.js

```
export async function getProducts(
  req,
  res,
  next
) {
  try {
    const products =
      await getAllProducts();

    res.json(products);
  } catch (error) {
    next(error);
  }
}
```

24. Измените getProductById

```
export async function getProductById(
  req,
  res,
  next
) {
  try {
    const id =
      Number(req.params.id);

    const product =
      await findProductById(id);

    if (!product) {
      return next(
        new AppError(
          "Product not found",
          404
        )
      );
    }

    res.json(product);
  } catch (error) {
    next(error);
  }
}
```

25. Самостоятельное задание: реализуйте categoryService.js

Создайте services/categoryService.js самостоятельно. Реализуйте асинхронную функцию getAllCategories(): получите через существующий pool все категории из таблицы categories, отсортируйте по id и верните массив строк результата. Используйте pool.query() и result.rows. Готовый код не приводится.

Подсказка: используйте в качестве образца getAllProducts() из productService.js. Для чтения категорий достаточно SELECT; JOIN здесь не нужен.

26. Самостоятельное задание: реализуйте categoryController.js

Создайте controllers/categoryController.js самостоятельно. Экспортируйте асинхронный обработчик getCategories(req, res, next), который вызывает getAllCategories(), возвращает массив категорий в JSON, а ошибки передаёт в next(error).

Подсказка: используйте структуру getProducts() из productController.js (try/catch, await, res.json, next). Готовый код обработчика не приводится.

27. Самостоятельное задание: реализуйте categoryRoutes.js

Создайте routes/categoryRoutes.js самостоятельно. Используйте Express Router, подключите getCategories к GET / внутри маршрутизатора и экспортируйте маршрутизатор по умолчанию.

Подсказка: ориентируйтесь на productRoutes.js. Итоговый внешний адрес должен быть GET /api/categories, а не GET /api/categories/categories.

28. Самостоятельное задание: подключите маршрут категорий в app.js

Самостоятельно импортируйте categoryRoutes в app.js и зарегистрируйте его на базовом пути /api/categories с помощью app.use().

Проверьте, что существующие маршруты /api/products и /api/products/:id продолжают работать.

Маршрут GET /api/categories должен проходить через Router → Controller → Service → PostgreSQL, а возможные ошибки — попадать в существующий централизованный errorHandler.

29. Проверьте productRoutes.js

Маршруты товаров должны по-прежнему обрабатывать получение всех товаров и одного товара по id.

```
router.get(
  "/",
  getProducts
);

router.get(
  "/:id",
  getProductById
);
```

30. Запустите сервер

```
npm start
```

Если в проекте используется другая команда запуска, используйте её.

31. Проверьте получение всех товаров

Выполните:

```
GET /api/products
```

Ожидаемый результат:

```
[
  {
    "id": 1,
    "name": "Keyboard",
    "price": 15000,
    "category": "Accessories",
    "inStock": true
  },
  {
    "id": 2,
    "name": "Mouse",
    "price": 8000,
    "category": "Accessories",
    "inStock": true
  },
  {
```

```
    "id": 3,
    "name": "Monitor",
    "price": 85000,
    "category": "Monitors",
    "inStock": false
  }
]
```

32. Проверьте получение одного товара

```
GET /api/products/2
```

Ожидается:

```
{
  "id": 2,
  "name": "Mouse",
  "price": 8000,
  "category": "Accessories",
  "inStock": true
}
```

33. Проверьте отсутствующий товар

```
GET /api/products/999
```

Ожидается статус 404 Not Found и ответ существующего обработчика ошибок:

```
{
  "message": "Product not found"
}
```

34. Проверьте получение категорий

```
GET /api/categories
```

Ожидаемый результат:

```
[
  {
    "id": 1,
    "name": "Accessories"
  },
  {
    "id": 2,
    "name": "Monitors"
  },
  {
    "id": 3,
```

```
    "name": "Components"
  }
]
```

35. Проверьте постоянное хранение

Через pgAdmin добавьте новый товар. Категория Components уже имеет id = 3.

```
INSERT INTO products (
  name,
  price,
  category_id,
  in_stock
)
VALUES (
  'SSD 2TB',
  65000,
  3,
  true
);
```

Затем снова выполните GET /api/products. Новый товар должен появиться в REST API.

36. Добавьте новую категорию и проверьте API

```
INSERT INTO categories (name)
VALUES ('Laptops');
```

Снова выполните GET /api/categories. В ответе должна появиться новая категория Laptops.

37. Перезапустите Node.js

Остановите Express-сервер сочетанием Ctrl + C, затем запустите его снова.

- Повторите GET /api/products.
- Повторите GET /api/categories.
- Добавленные данные должны сохраниться после перезапуска Node.js.

Это демонстрирует отличие постоянного хранения в PostgreSQL от временного массива в памяти процесса Node.js.

38. Проведите эксперимент с ошибкой подключения

Временно измените в .env:

```
PGPORT=5433
```

Перезапустите сервер и выполните запрос GET /api/products или GET /api/categories. Ошибка подключения к PostgreSQL должна быть передана в существующий errorHandler.

После эксперимента верните правильный порт:

```
PGPORT=5432
```

Итоговая структура проекта

```
express-shop/
├── config/
│   └── db.js
├── controllers/
│   ├── categoryController.js
│   └── productController.js
├── errors/
├── middleware/
├── routes/
│   ├── categoryRoutes.js
│   └── productRoutes.js
├── services/
│   ├── categoryService.js
│   └── productService.js
├── validators/
├── public/
├── .env
├── .gitignore
├── app.js
├── package.json
└── package-lock.json
```

Требования к результату

- Приложение должно использовать PostgreSQL вместо массива для чтения товаров.
- База данных должна называться shop_db.
- База данных должна содержать таблицы products и categories.
- Таблица products должна ссылаться на categories через поле category_id.
- Приложение должно подключаться к PostgreSQL через пакет pg.
- Для соединений должен использоваться Pool.
- Параметры подключения должны храниться в .env.
- Подключение к базе должно быть вынесено в config/db.js.
- SQL-запросы должны выполняться через pool.query().
- Результаты SELECT должны читаться через result.rows.
- Поиск товара по id должен использовать параметр $1.
- GET /api/products должен возвращать список товаров.
- GET /api/products/:id должен возвращать один товар или 404.
- GET /api/categories должен возвращать список категорий.
- Формат товара в REST API должен сохранить поле category с названием категории.
- Для получения названия категории товаров должен использоваться JOIN.
- Ошибки PostgreSQL должны передаваться в существующий errorHandler.
- Данные должны сохраняться после перезапуска Node.js.
- CREATE TABLE categories студент составляет самостоятельно, без готового SQL-шаблона.
- Реализацию GET /api/categories (Service, Controller, Router и подключение в app.js) студент пишет самостоятельно.

Контрольные вопросы

1. Зачем категории вынесены в отдельную таблицу?
2. Что хранится в products.category_id?
3. Для чего используется внешний ключ REFERENCES categories(id)?
4. Зачем в запросе товаров используется JOIN?

5. Почему API продолжает возвращать поле category, хотя в products хранится category_id?
6. Чем result.rows отличается от result.rows[0]?
7. Почему запрос по id должен использовать $1?
8. Какой путь проходит запрос GET /api/categories через слои приложения?
9. Почему pool.end() не следует вызывать после каждого HTTP-запроса?
10. Почему данные PostgreSQL сохраняются после перезапуска Node.js?